kernal function explain all concept

🔥 Kernel Function – Complete Concept (Easy → Advanced)

Kernel is mostly used in **Support Vector Machine (SVM)**.

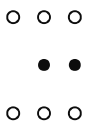
👉 Main idea:
Kernel helps us handle **non-linear data** by converting it into **higher dimension** so that it becomes linearly separable.

1️⃣ Why Kernel is Needed?

In SVM, we try to draw a **straight line (hyperplane)** to separate classes

ChatGPT

Code



This data is **not linearly separable** in 2D.

So what do we do?

👉 We move data into **higher dimension**

Example:
2D → 3D

Now it becomes separable by a plane.

This trick is called:

🎯 Kernel Trick

Instead of actually converting data into higher dimension (which is expensive),

We use a **kernel function** to directly compute similarity in higher dimension.

2️⃣ What is Kernel Function (Mathematically)?

Kernel function:

$$K(x, y) = \varphi(x) \cdot \varphi(y)$$

Where:

- x, y = input vectors
- $\varphi(x)$ = mapping to higher dimension
- Dot product done in higher space

👉 But we compute it **without explicitly calculating φ(x)**.

That's why it's called **Kernel Trick**.

3️⃣ Types of Kernel Functions

1️⃣ Linear Kernel

$$K(x, y) = x \cdot y$$

✔ Used when data is already linearly separable
✔ Fast
✔ No transformation

Used in:

- Text classification
- High dimensional sparse data

---

2 Polynomial Kernel

$$K(x, y) = (x \cdot y + c)^d$$

Where:

- d = degree
- c = constant

✔ Handles curved boundaries
✔ More flexible than linear

If degree = 2 → quadratic boundary
If degree = 3 → cubic boundary

---

3 RBF (Radial Basis Function) Kernel (Most Important)

$$K(x, y) = exp(-\gamma||x - y||^2)$$

Where:

- γ (gamma) controls spread

✔ Most powerful
✔ Handles complex non-linear data
✔ Default choice in SVM

💡 If gamma is small → smooth boundary
💡 If gamma is large → overfitting risk

---

4 Sigmoid Kernel

$$K(x, y) = tanh(\alpha x \cdot y + c)$$

✔ Looks like neural network
✔ Less commonly used

---

4 Geometric Meaning

Kernel measures:

👉 **Similarity between two points**

If:

- K(x, y) is large → points are similar
- K(x, y) is small → points are far

Especially in RBF:

- Close points → value near 1
- Far points → value near 0

5️⃣ Hyperparameters in Kernel

Important in SVM:

Parameter	Meaning
C	Regularization (overfitting control)
Gamma	Spread of RBF
Degree	Polynomial degree

6️⃣ Kernel Trick Intuition Example

Suppose:

x = (x1, x2)

We map to:

φ(x) = ($x1^2$, √2 x1x2, $x2^2$)

Instead of computing φ(x), we directly use:

$$(x \cdot y)^2$$

Boom 💥

Same result, less computation.

7️⃣ When to Use Which Kernel?

✔ Linear → Large features, simple boundary

✔ Polynomial → Moderate complexity

✔ RBF → Complex non-linear data (default choice)

✔ Sigmoid → Rare

8️⃣ Kernel in Classification, Regression, Outlier Detection

Kernel used in:

- SVM Classification
- SVR (Support Vector Regression)
- One-Class SVM (Outlier detection)

9️⃣ Advanced Concept

Mercer's Theorem

For a function to be valid kernel:

It must produce a **positive semi-definite matrix**.

That matrix is called:

👉 Kernel Matrix (Gram Matrix)

$$K_{ij} = K(x_i, x_j)$$

🔟 Overfitting in Kernel

If:

- Gamma too high

- Degree too high
- C too large

Model becomes too complex → overfitting.

🎯 Final Summary

Kernel = similarity function

Kernel Trick = compute high dimension dot product without mapping

RBF = most powerful

Linear = fastest

Polynomial = medium complexity

If you want, I can next explain:

✅ Kernel matrix with example
✅ RBF intuition with diagram explanation
✅ Kernel in regression deeply
✅ Code example in Python
✅ Mathematical derivation step-by-step

Tell me which level you want:
Basic / Interview / Mathematical / Coding 🔥

svm

🔥 Support Vector Machine (SVM) — Complete Concept (Easy → Advanced)

SVM is a **supervised machine learning algorithm** used for:

- ✅ Classification
- ✅ Regression

- ✅ Outlier Detection

Main goal:

👉 **Find the best boundary that separates data with maximum margin.**

1️⃣ Basic Idea of SVM (Intuition)

Imagine two classes of points:

🔵 Class A
🔴 Class B

Many lines can separate them.
But SVM chooses the line that:

👉 **Maximizes the margin (distance between classes).**

That best line is called:

🎯 Optimal Hyperplane

2️⃣ What is Margin?

Margin = Distance between:

- Hyperplane
- Closest data points from each class

The closest points are called:

⭐ Support Vectors

These points "support" the boundary.

Only support vectors decide the hyperplane.

3️⃣ Hard Margin vs Soft Margin

🔹 Hard Margin SVM

Used when:

- Data is perfectly separable
- No noise

Problem:

- Not practical in real world

🔹 Soft Margin SVM

Used when:

- Data has noise
- Some misclassification allowed

We introduce:

🎯 C (Regularization Parameter)

- Large C → Low error, small margin (overfitting risk)
- Small C → Larger margin, allow mistakes (better generalization)

4️⃣ Mathematical Formulation (Classification)

Hyperplane equation:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- b = bias

Goal:

$$\text{Maximize margin} = \frac{2}{||w||}$$

Equivalent to minimizing:

$$\frac{1}{2}||w||^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

5️⃣ What if Data is Not Linearly Separable?

👉 Use **Kernel Trick**

Kernel maps data to higher dimension.

Common kernels:

- Linear
- Polynomial
- RBF (Most used)
- Sigmoid

RBF formula:

$$K(x, y) = exp(-\gamma||x - y||^2)$$

6️⃣ SVM for Regression (SVR)

Instead of separating classes,

We fit a function inside a margin.

Concept:

ε (Epsilon tube)

Model ignores errors within ε distance.

Goal:

Minimize:

- Flatness
- Errors outside epsilon tube

7️⃣ SVM for Outlier Detection

Used in:

👉 One-Class SVM

Idea:

- Learn boundary around normal data
- Points outside → anomalies

8️⃣ Geometric Understanding

SVM tries to:

✔ Push boundary far from both classes
✔ Use only important points (support vectors)
✔ Ignore unnecessary data

This makes it:

- Memory efficient
- Powerful in high dimension

9️⃣ Advantages of SVM

✔ Works well in high dimensions
✔ Effective with small dataset
✔ Strong theoretical foundation
✔ Uses only support vectors

🔟 Disadvantages

❌ Slow for very large dataset
❌ Choosing kernel is tricky
❌ Hard to interpret

1️⃣1️⃣ Important Hyperparameters

Parameter	Meaning
C	Regularization strength
Kernel	Type of transformation
Gamma	RBF spread
Degree	Polynomial degree
Epsilon	SVR tolerance

1️⃣2️⃣ When to Use SVM?

Use SVM when:

- ✔ Dataset small-medium
- ✔ High dimension (text data)
- ✔ Clear margin separation
- ✔ Non-linear boundary needed

Avoid when:

- ❌ Very large dataset
- ❌ Need probabilistic output (use Logistic Regression instead)

🎯 Final Simple Definition

SVM =

"A machine learning algorithm that finds the best boundary by maximizing margin using support vectors."

If you want next level, tell me:

- 🔥 Mathematical derivation (Lagrange multipliers)
- 🔥 Dual form explanation
- 🔥 Kernel matrix deep theory
- 🔥 Coding in Python
- 🔥 Interview questions on SVM

linearly seprable data

🔥 Linearly Separable Data — Complete Concept (Easy → Advanced)

1️⃣ Simple Definition

Linearly separable data means:

👉 We can separate two classes using a **straight line** (in 2D)
👉 Or a **plane** (in 3D)
👉 Or a **hyperplane** (in higher dimensions)

2️⃣ Easy Example (2D)

Imagine points like this:

```
🔴 🔴 🔴        (Class A)

        🔵 🔵 🔵    (Class B)
```

You can draw a **single straight line** between them.

✔ That dataset is **linearly separable**.

---

3 Mathematical Meaning

Data is linearly separable if there exists:

$$w \cdot x + b = 0$$

Such that:

$$y_i(w \cdot x_i + b) > 0$$

Where:

- w = weight vector
- b = bias
- y_i = class label (+1 or -1)

If this condition is satisfied for all points →

✅ Data is linearly separable.

---

4 Geometry Understanding

In:

- 2D → Separation by a **line**
- 3D → Separation by a **plane**
- nD → Separation by a **hyperplane**

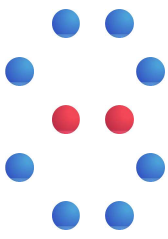
Hyperplane equation:

$$w \cdot x + b = 0$$

---

5 Example of Non-Linearly Separable Data

Code



No straight line can separate this.

This requires:

👉 Feature transformation
👉 Kernel Trick in SVM

6️⃣ Why It Matters in SVM?

In **Support Vector Machine (SVM)**:

- If data is linearly separable → use **Hard Margin SVM**
- If not → use **Soft Margin + Kernel**

SVM tries to find the **maximum margin hyperplane**.

7️⃣ Conditions for Linear Separability

Data is linearly separable if:

✔️ Classes do not overlap
✔️ There exists at least one hyperplane separating them
✔️ No point lies on wrong side

8️⃣ Practical Example

Example 1: Height vs Weight

If:

- All males have higher height & weight
- All females have lower height & weight

Then likely linearly separable.

Example 2: XOR Problem

XOR is **not linearly separable**.

That's why simple linear model fails.

Neural networks or kernels are required.

9️⃣ Visual Intuition

Imagine:

You put a ruler between two groups of objects.

If one straight placement works → linearly separable
If you must curve the ruler → not linearly separable

🔟 Real-World Meaning

Linearly separable data is:

- ✔ Easier to model
- ✔ Faster to train
- ✔ Less computationally expensive

Non-linear data requires:

- Kernel methods
- Polynomial features
- Neural networks

🎯 Final Short Definition

Linearly separable data =
"Data that can be separated by a straight line (or hyperplane) without error."

If you want next:

- 🔥 Proof of linear separability
- 🔥 XOR mathematical explanation

- 🔥 Visual geometric explanation
- 🔥 Coding example
- 🔥 Hard vs Soft margin deeply

Tell me your level 🚀

Mathematical derivation (Lagrange multipliers)

🔥 Dual form explanation

🔥 Kernel matrix deep theory

🔥 Coding in Python explain all

🔥 Interview questions on SVM

Perfect 👍
Below is **SVM from Mathematical → Dual → Kernel Matrix → Python Code → Interview (🔥 exam + interview ready)**

🔥 1️⃣ Mathematical Derivation of SVM (Lagrange Multipliers)

👉 Problem Setup (Binary Classification)

Given training data:

$$(x_i, y_i), \quad y_i \in \{-1, +1\}$$

Hyperplane:

$$w \cdot x + b = 0$$

🎯 Goal of SVM

Maximize **margin**:

$$\text{Margin} = \frac{2}{||w||}$$

Maximizing margin ⇔ Minimizing:

$$\frac{1}{2}||w||^2$$

🔹 Hard Margin Optimization Problem

$$\min_{w,b} \frac{1}{2}||w||^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

🔥 Convert to Lagrangian

Introduce **Lagrange multipliers** $\alpha_i \geq 0$

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_i \alpha_i \left[y_i(w \cdot x_i + b) - 1\right]$$

🔹 Take Partial Derivatives

1️⃣ w.r.t w

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$$

2️⃣ w.r.t b

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$$

🔥 2️⃣ Dual Form Explanation (MOST IMPORTANT)

👉 Why Dual Form?

- Depends only on **dot products**
- Allows **kernel trick**
- Faster when features > samples

🎯 Dual Optimization Problem

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

Subject to:

$$\alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$$

⭐ Key Insight

- Only points with $\alpha_i > 0$ matter
- These are **Support Vectors**

🔹 Decision Function

$$f(x) = \sum_i \alpha_i y_i (x_i \cdot x) + b$$

🔥 3️⃣ Kernel Matrix (Deep Theory)

🎯 Kernel Trick

Replace dot product:

$$x_i \cdot x_j \Rightarrow K(x_i, x_j)$$

🔹 Kernel Matrix (Gram Matrix)

For dataset with n points:

$$K = \begin{bmatrix} K(x_1, x_1) & K(x_1, x_2) & \dots \\ K(x_2, x_1) & K(x_2, x_2) & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Properties:

- ✔ Symmetric
- ✔ Positive Semi-Definite
- ✔ Required by **Mercer's Theorem**

🔹 Common Kernels

Linear

$$K(x, y) = x \cdot y$$

Polynomial

$$K(x, y) = (x \cdot y + c)^d$$

RBF (Gaussian)

$$K(x, y) = e^{-\gamma ||x-y||^2}$$

🔥 Why Kernel Matrix is Powerful?

- Avoids explicit feature mapping
- Works in infinite dimensional space (RBF)
- Only similarity matters

🔥 4️⃣ SVM Coding in Python (Step-by-Step)

Classification (Linear & RBF)

Python

```python
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Dataset
X, y = make_classification(n_samples=100, n_features=2,
                           n_classes=2, random_state=42)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y)

# SVM model
model = SVC(kernel='rbf', C=1, gamma=0.5)

# Training
model.fit(X_train, y_train)

# Prediction
print(model.score(X_test, y_test))
```

Support Vectors

Python

```python
print(model.support_vectors_)
print(model.n_support_)
```

SVM Regression (SVR)

Python

```python
from sklearn.svm import SVR
```

```
svr = SVR(kernel='rbf', C=100, epsilon=0.1)
svr.fit(X_train, y_train)
```

🔹 One-Class SVM (Outlier Detection)

Python

```
from sklearn.svm import OneClassSVM

ocsvm = OneClassSVM(kernel='rbf', nu=0.05)
ocsvm.fit(X_train)
```

🔥 5️⃣ Interview Questions on SVM (WITH ANSWERS)

Q1️⃣ Why SVM is called maximum margin classifier?

👉 Because it chooses the hyperplane that maximizes distance from nearest data points.

Q2️⃣ What are support vectors?

👉 Data points closest to hyperplane that determine the decision boundary.

Q3️⃣ Why dual formulation is preferred?

👉 It enables kernel trick and depends only on dot products.

Q4️⃣ What happens if C is very large?

👉 Model tries to classify all points correctly → overfitting.

Q5️⃣ Why RBF kernel is popular?

👉 It maps data into infinite dimensional space and handles complex patterns.

Q6️⃣ Can SVM give probability output?

👉 No (by default). Needs Platt scaling.

Q7️⃣ Is SVM sensitive to feature scaling?

👉 YES. Always scale features.

Q8️⃣ Difference between Logistic Regression & SVM?

Logistic	SVM
Probabilistic	Margin-based
Uses all points	Uses support vectors
Faster on large data	Strong on small data

Q9️⃣ What is Mercer's theorem?

👉 Condition that ensures a kernel is valid (positive semi-definite).

Q🔟 When NOT to use SVM?

👉 Very large datasets, noisy data, need interpretability.

🎯 FINAL MASTER SUMMARY

✔ SVM maximizes margin
✔ Lagrange → Dual → Kernel trick
✔ Kernel matrix enables non-linearity

- ✔ Only support vectors matter
- ✔ RBF = safest default
- ✔ Strong theory + strong performance

If you want next 🔥:

- 📐 Numerical example solved fully
- 🎨 Decision boundary visualization
- 🧠 SVM vs Neural Network
- 🧮 Solving optimization step-by-step
- 📄 PDF notes for revision

Just tell me 🚀

1/2

RBF

🔥 RBF (Radial Basis Function) Kernel — Complete Concept (Easy → Advanced)

RBF is the **most powerful and most used kernel** in SVM.

1️⃣ What is RBF?

RBF stands for:

👉 **Radial Basis Function**

Most common form is **Gaussian Kernel**.

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

Where:

- x, y = two data points

- $||x - y||^2$ = squared Euclidean distance
- γ (gamma) = spread parameter

2️⃣ Intuition (Very Important)

RBF measures:

👉 **How close two points are**

- If two points are very close → distance small → kernel ≈ 1
- If far apart → distance large → kernel ≈ 0

So RBF = **similarity function**

3️⃣ Why is it called "Radial"?

Because value depends only on:

$$||x - y||$$

Distance from center (radial distance).

If you draw circles around a point:

- Same distance → same kernel value

Like ripples in water 🌊

4️⃣ Role of Gamma (γ)

Gamma controls how far influence of one point reaches.

🔹 Small Gamma

- Wide influence
- Smooth decision boundary
- High bias
- Underfitting risk

🔹 Large Gamma

- Narrow influence
- Complex boundary
- High variance
- Overfitting risk

5️⃣ Why RBF is So Powerful?

Because:

👉 It maps data to **infinite dimensional space**

But we don't compute that directly.

We use:

⭐ Kernel Trick

Replace dot product with:

$$K(x_i, x_j)$$

No explicit transformation required.

6️⃣ Geometric Meaning

In linear SVM:

Boundary = straight line

In RBF SVM:

Boundary can become:

- Curved
- Wavy
- Complex

Perfect for:

- ✔ Non-linear data
- ✔ Complex patterns
- ✔ Real-world messy datasets

7️⃣ Mathematical Insight (Advanced)

Decision function:

$$f(x) = \sum_i \alpha_i y_i \exp(-\gamma \|x_i - x\|^2) + b$$

Each support vector creates a **Gaussian bump**.

Final boundary = combination of all bumps.

8️⃣ Relation Between C and Gamma

Two most important parameters:

Parameter	Controls
C	Margin vs Misclassification
Gamma	Complexity of boundary

🎯 Case Analysis

- High C + High Gamma → Overfitting
- Low C + Low Gamma → Underfitting
- Moderate C + Moderate Gamma → Good model

9️⃣ When to Use RBF?

Use RBF when:

✔ Data is non-linearly separable
✔ No prior knowledge about feature interaction
✔ Medium dataset size

Avoid when:

❌ Very large dataset (slow)
❌ Features not scaled

🔟 Why Feature Scaling is IMPORTANT?

Because RBF uses:

$$||x - y||^2$$

If one feature has large scale → dominates distance.

So always use:

- StandardScaler
- MinMaxScaler

1️⃣1️⃣ Python Example

Python

```python
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.datasets import make_classification

# Data
X, y = make_classification(n_samples=200, n_features=2, random_state=42)

# Model
model = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1, gamma=0.5)
```

```
)

model.fit(X, y)
```

1️⃣ 2️⃣ RBF vs Polynomial

RBF	Polynomial
Infinite dimension	Finite dimension
More flexible	Controlled complexity
Default choice	Requires degree tuning

🎯 Final Simple Definition

RBF Kernel =

"A similarity function that measures distance between two points and creates flexible non-linear decision boundaries."

If you want next 🔥:

- 🎨 RBF decision boundary visualization
- 🧮 Numerical example step-by-step
- 🧠 RBF vs Neural Network comparison
- 📐 Gamma effect graph explanation
- 💯 Interview deep questions on RBF

Tell me your level 🚀

ROC AND AUC CURVE

🔥 ROC and AUC Curve — Complete Concept (Easy → Advanced)

Used in:

👉 Binary Classification
👉 Model evaluation
👉 Comparing classifiers

1️⃣ Confusion Matrix First (Very Important)

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Where:

- TP = True Positive
- FP = False Positive
- TN = True Negative
- FN = False Negative

2️⃣ What is ROC Curve?

ROC = **Receiver Operating Characteristic**

It is a graph between:

$$\textbf{TPR (True Positive Rate)} \quad vs \quad \textbf{FPR (False Positive Rate)}$$

🔹 TPR (Recall / Sensitivity)

$$TPR = \frac{TP}{TP + FN}$$

Measures:

👉 How many actual positives we correctly predicted.

🔹 FPR

$$FPR = \frac{FP}{FP + TN}$$

Measures:

👉 How many actual negatives we wrongly predicted as positive.

3️⃣ Why ROC is Needed?

Most classifiers output:

👉 Probability (0 to 1)

To convert probability into class label:

We choose a threshold (usually 0.5).

But changing threshold changes:

- TP
- FP
- FN
- TN

ROC shows performance across **all possible thresholds**.

4️⃣ How ROC Curve is Created

1. Sort probabilities
2. Change threshold from 1 → 0
3. Compute TPR & FPR
4. Plot curve

5️⃣ Ideal ROC Curve

Perfect model:

- TPR = 1
- FPR = 0

Graph goes straight up then across.

Worst model:

Diagonal line.

That line represents:

👉 Random guessing

6️⃣ What is AUC?

AUC = **Area Under the Curve**

It measures overall performance.

$$0 \leq AUC \leq 1$$

🔹 Interpretation

AUC	Meaning
1.0	Perfect classifier
0.9 – 1.0	Excellent
0.8 – 0.9	Good
0.7 – 0.8	Fair
0.5	Random guessing
< 0.5	Worse than random

7️⃣ Probabilistic Meaning of AUC

AUC =

👉 Probability that model ranks a random positive higher than a random negative.

8️⃣ Mathematical Insight

AUC equals:

$$P(score_{positive} > score_{negative})$$

It's equivalent to:

- Mann–Whitney U statistic

9️⃣ ROC vs Accuracy

Accuracy depends on threshold.

ROC:

✔ Threshold independent
✔ Works better for imbalanced data

🔟 ROC vs Precision-Recall Curve

ROC	Precision-Recall
Good for balanced data	Better for imbalanced data
Uses TPR & FPR	Uses Precision & Recall

For highly imbalanced dataset:

👉 Prefer Precision-Recall curve.

1️⃣1️⃣ Python Example

Python

```python
from sklearn.metrics import roc_curve, auc
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Dataset
X, y = make_classification(n_samples=500, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y)

# Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Probabilities
y_scores = model.predict_proba(X_test)[:,1]

# ROC
fpr, tpr, thresholds = roc_curve(y_test, y_scores)
roc_auc = auc(fpr, tpr)

# Plot
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()

print("AUC:", roc_auc)
```

1️⃣2️⃣ When to Use ROC-AUC?

Use when:

✔ Binary classification
✔ Want threshold-independent evaluation
✔ Compare multiple models

Avoid when:

❌ Highly imbalanced dataset (use PR curve instead)

1️⃣3️⃣ Interview Questions

Q1: Why ROC curve starts at (0,0)?

Because threshold = 1 → no positives predicted.

Q2: Why diagonal line means random?

Because TPR = FPR at all thresholds.

Q3: Can AUC be >1?

No.

Q4: What does AUC = 0.5 mean?

Model has no discrimination power.

Q5: Why ROC is insensitive to class imbalance?

Because it uses rate, not absolute counts.

🎯 Final Simple Definition

ROC curve =

"Graph showing trade-off between true positive rate and false positive rate at all thresholds."

AUC =

"Single number measuring overall ranking ability of classifier."

If you want next:

- 🔥 Numerical example step-by-step
- 🔥 ROC for SVM specifically
- 🔥 Multiclass ROC
- 🔥 Mathematical derivation
- 🔥 Visualization with explanation

Tell me your level 🚀